HomePilot AI - Complete Hackathon Product & Submission Package

A household coordination agent for reducing family mental load

Prepared for Build, Ship, Shape: Amazon Developer Hackathon 2026

Working title note: "HomePilot" is used as the hackathon codename in this package. Public app-name clearance is recommended before launch because similar names already exist in software/smart-home markets.

Executive summary
HomePilot AI is a household coordination agent that turns messy family conversations into a shared, feasible plan. Instead of asking every family member to maintain another calendar or task app, HomePilot listens to natural-language requests, understands constraints, detects conflicts, delegates responsibility, asks for confirmation when needed, and surfaces only the next actions each person needs.

1. Product concept

1.1 Problem

Households often have information spread across memory, messaging apps, calendars, shopping lists, school notices, and sticky notes. The hard part is not merely storing tasks; it is the invisible cognitive work of anticipating needs, remembering deadlines, deciding who should do what, checking conflicts, and following up. Research on cognitive household labor describes this planning and anticipation as a distinct burden, while recent community discussions show that shared calendars and family task apps often fail because they require sustained manual upkeep.

1.2 Solution

HomePilot AI is a household coordination agent that turns messy family conversations into a shared, feasible plan. Instead of asking every family member to maintain another calendar or task app, HomePilot listens to natural-language requests, understands constraints, detects conflicts, delegates responsibility, asks for confirmation when needed, and surfaces only the next actions each person needs.

1.3 Value proposition

HomePilot reduces the need for a human "household project manager". The product does not replace family decisions; it makes the coordination work explicit, shared, and actionable.

1.4 One-line pitch

HomePilot AI turns household chaos into a coordinated plan - without forcing the family to maintain yet another productivity app.

1.5 Why this is different

- Most family apps are databases: users must manually enter and maintain every event or task. HomePilot is an agent: it converts unstructured family context into structured actions.

- The core unit is not a to-do item but a household commitment with an owner, deadline, dependencies, constraints and confirmation state.
- The product focuses on reducing cognitive labor: anticipating, deciding, delegating and following up - not just reminding.
- The MVP is intentionally built around a reproducible simulated household, making the 3-minute demo reliable while still showing real MCP/AWS execution.

2. Evidence of user need

Academic research distinguishes cognitive household labor from physical household labor and links the planning/anticipation burden to stress and family-work conflict. A 3,000-parent US study identified distinct forms of domestic cognitive labor, while a separate study of 322 mothers found cognitive labor especially disproportionate and associated with stress, burnout and relationship outcomes. Community discussions in 2026 also repeatedly describe failed shared calendars, abandoned family task apps and one person becoming the de facto household coordinator.

Product insight
The pain point is not "we need another calendar." The pain point is "someone still has to think for the calendar." HomePilot targets that thinking work.

3. Target users and jobs-to-be-done

Persona	Situation	Job to be done
Primary coordinator parent	Works full-time, remembers school, appointments, shopping and deadlines	Reduce mental load without creating another system to maintain
Partner / co-parent	Wants to help but lacks the full context	See ownership clearly and receive only relevant next actions
Teen / older child	Has school, activities and chores	Know what is expected without reading long family chats
Caregiver family member	Coordinates an older relative across siblings	Avoid missed appointments, duplicated errands and unclear ownership

4. Core user stories

- As a parent, I can describe several obligations in one message and get a feasible plan instead of manually creating five separate tasks.
- As a co-parent, I can see exactly what I own and why, without needing to inspect the entire family calendar.
- As a caregiver, I can prevent duplicated errands and missed appointments by sharing one current plan across siblings.
- As a family member, I can reject or modify a proposed assignment before the agent commits it.
- As a household, we can ask "what is at risk today?" and receive a short exception-focused answer rather than a full data dump.

5. Feature set

Feature	What it does
Natural-language household intake	One message can contain multiple people, tasks, deadlines,

Feature	What it does
	dependencies and constraints.
Constraint-aware planning	Detects calendar collisions, travel-time issues, overloaded family members and missing ownership.
Delegation with confirmation	Proposes who should own a task; significant schedule changes require user approval.
Shared household state	Tasks, events, shopping needs and commitments live in one structured model.
Role-specific next actions	Each family member sees a small, relevant list instead of the full household backlog.
Daily brief	A concise morning/evening summary of what changed, what is at risk and what needs attention.
Conflict resolver	When plans collide, suggests options with tradeoffs instead of silently changing schedules.
Preference memory	Learns non-sensitive preferences such as preferred pickup person, quiet hours and notification style.
Explainable actions	Shows why an assignment or reschedule was proposed and which constraints were considered.
Demo-safe simulation	Uses synthetic household data so the hackathon demo is reproducible and privacy-safe.

6. Golden demo scenario

Scenario: Friday evening. The family has a pediatric appointment Saturday at 09:00, one parent starts work at 08:30, the other has a class at 10:00, groceries are low, and a school project is due Monday. The user speaks or types the situation once. HomePilot extracts the commitments, checks availability, proposes that the second parent handles the appointment, schedules grocery pickup on the return route, assigns the school-project check to the first parent in the evening, and asks for confirmation before saving the plan.

1. User: "Tomorrow our child has the doctor at 9. I start work at 8:30, Ali has class at 10, we are out of milk, and the science project is due Monday."
2. HomePilot identifies four obligations, two people, hard time constraints and one shopping dependency.
3. The agent calls household and calendar MCP tools.
4. It detects that the first parent cannot attend the appointment and that Ali has enough travel buffer before class.
5. It proposes a plan and explicitly explains the tradeoff.
6. User confirms. Tasks/events are updated and each family member gets only their relevant next action.

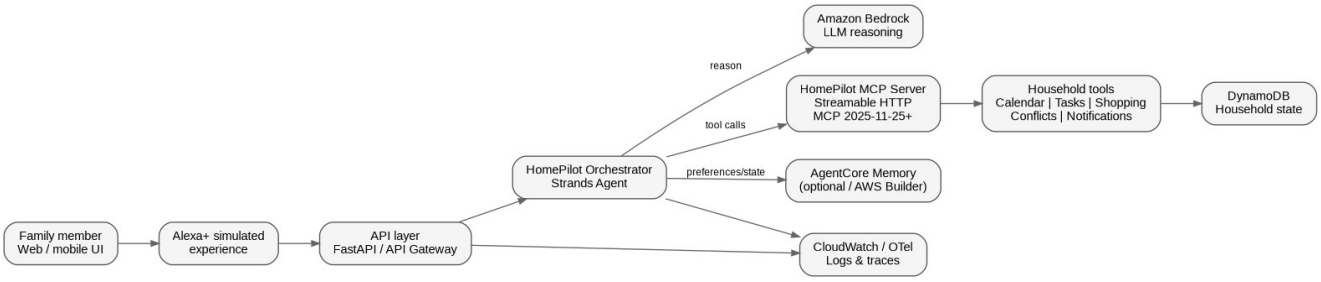
7. UX principles

- Conversation-first, dashboard-second: the fastest path is natural language; the dashboard is for verification and adjustment.
- Exception-focused: highlight conflicts, unowned tasks and overdue commitments instead of showing everything equally.
- Progressive confirmation: low-risk drafting can be automatic; external or schedule-changing actions require explicit approval.
- Low notification burden: one actionable brief is better than ten reminders.
- Visible reasoning: show the constraints behind a recommendation in plain language.
- Family dignity: never rank family members, shame missed tasks, or frame household contribution as a competition.

8. Screen inventory

Screen	Purpose	Key components
Today	Household control center	At-risk items, next actions, daily brief, unresolved conflicts
Ask HomePilot	Natural-language intake	Chat input, suggestion chips, confirmation cards
Plan	Shared structured plan	Timeline, owner badges, deadlines, dependencies
People	Family roles & preferences	Availability, notification preference, responsibility areas
Tasks	Operational details	Status, owner, due date, source, reasoning
Shopping	Shared errands	Items, assignee, optional route grouping
Activity	Audit trail	Agent proposals, confirmations, edits, tool calls
Settings	Privacy and control	Data retention, integrations, quiet hours, export/delete

9. Technical architecture



Recommended hackathon architecture: a web-based simulated Alexa+ experience connects to a Python FastAPI backend. A Strands agent uses Amazon Bedrock for reasoning and invokes a self-hosted HomePilot MCP server over Streamable HTTP. Household state is stored in DynamoDB; optional Amazon Bedrock AgentCore Memory can retain preferences and session summaries. CloudWatch/OpenTelemetry captures traceable tool activity. This architecture also gives the project a credible AWS Builder mini-challenge story.

9.1 Primary track strategy

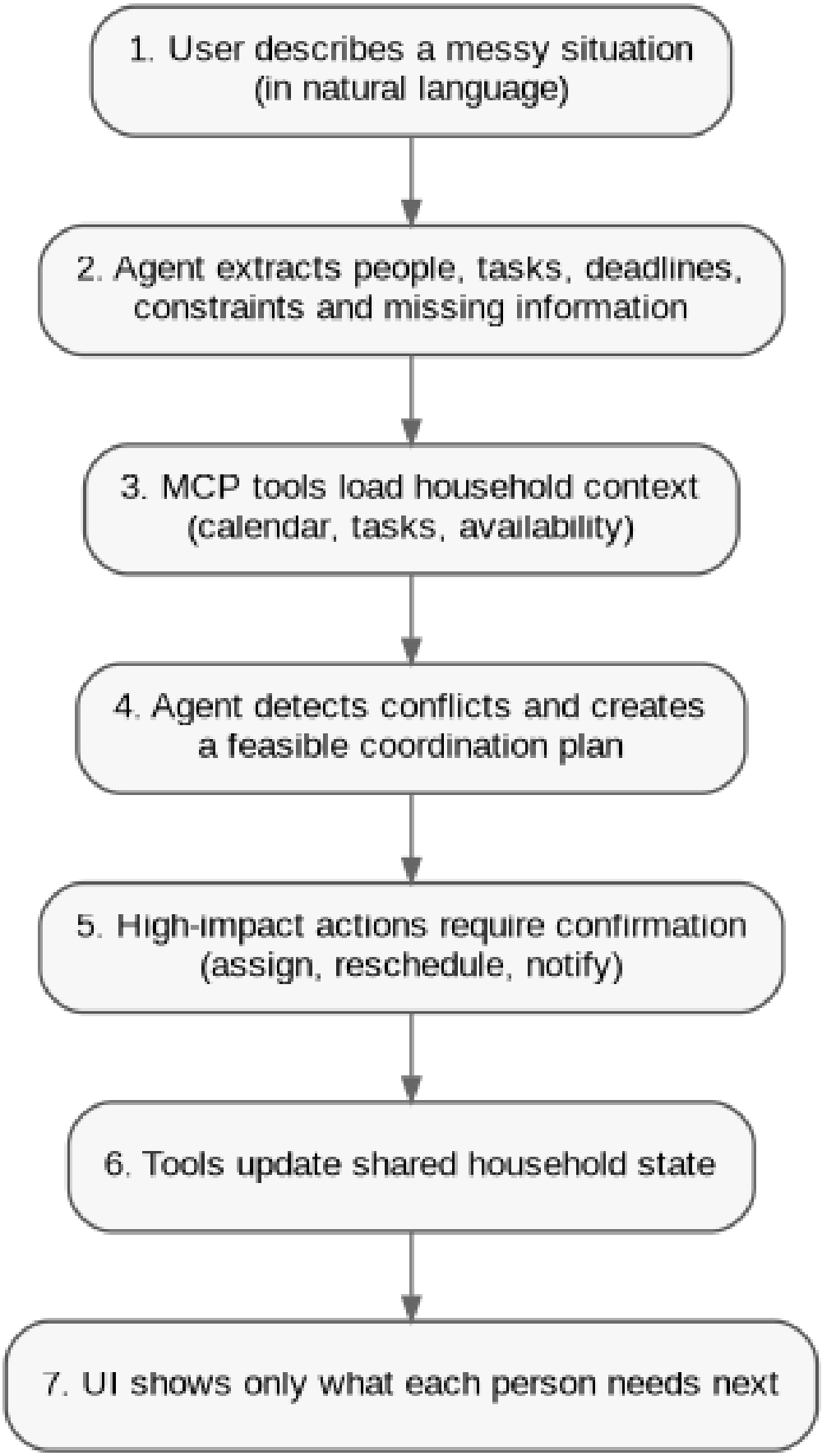
- Primary track: Alexa+.
- Preferred compliance path: working self-hosted MCP server implementing MCP 2025-11-25 or later over Streamable HTTP, demonstrated through the project web UI.
- Fallback path if MCP integration slows development: official rules also allow a simulated Alexa+ experience built with any agentic tool; however, the MCP path gives a stronger technical narrative.
- Mini challenge target: AWS Builder using Bedrock + Strands + AgentCore/DynamoDB/CloudWatch as a multi-service agentic system.
- Optional Open Source mini challenge: publish a reusable household-coordination MCP schema/toolkit or a generic conflict-resolution component with tests.

9.2 MCP tool contract

Tool	Purpose

get_household_context	Return family members, roles, preferences and current household state.
list_events	Read upcoming events and availability windows.
create_event	Create a proposed event after confirmation.
list_tasks	Read active tasks with owner, deadline and status.
create_task	Create a structured household task.
assign_task	Assign or reassign a task with a reason.
detect_conflicts	Find schedule, ownership and deadline conflicts.
propose_plan	Generate a candidate coordination plan from constraints.
shopping_list	Read/add/complete shopping items.
send_notification	Send a simulated or real notification to a family member.
daily_brief	Return only high-priority items for the day.

9.3 Agent execution flow


1. User describes a messy situation (in natural language)
2. Agent extracts people, tasks, deadlines, constraints and missing information
3. MCP tools load household context (calendar, tasks, availability)
4. Agent detects conflicts and creates a feasible coordination plan
5. High-impact actions require confirmation (assign, reschedule, notify)
6. Tools update shared household state
7. UI shows only what each person needs next

9.4 Suggested repository structure

```
homepilot-ai/
  apps/web/                 # Next.js/React simulator UI
  services/api/             # FastAPI gateway
  services/agent/           # Strands orchestrator, prompts, policies
  services/mcp/             # MCP 2025-11-25+ Streamable HTTP server
  packages/domain/          # household data model + conflict engine
  infra/                    # CDK/Terraform/CloudFormation
  demo/                     # deterministic synthetic scenario data
  tests/                    # unit, integration, agent evals
  docs/                     # architecture, demo, feedback, friction log
  LICENSE
  README.md
```

10. Domain model

Entity	Key fields
Household	id, name, timezone, quiet_hours, preferences
Person	id, display_name, role, availability, notification_style
Commitment	id, type, title, start/end or due_at, owner, priority, source
Constraint	person, time window, hard/soft, reason
Task	status, owner, due_at, dependencies, effort estimate
ShoppingItem	name, quantity, priority, assigned_to, status
AgentProposal	proposed actions, rationale, confidence, confirmation_required
AuditEvent	timestamp, actor, tool, before/after, approval state

11. Agent policy and guardrails

- Never silently cancel or reschedule a family member's commitment.
- Require confirmation before sending external notifications, creating high-impact events, spending money, or sharing sensitive information.
- For ambiguous ownership, propose options rather than inventing consent.
- Treat health-related entries such as "take medicine" as reminders only; do not provide diagnosis, dosage or medical advice.
- Use minimum necessary household data and synthetic data in the public demo.
- Record an audit event for each agent tool call that changes state.
- Allow undo for all reversible actions in the MVP.

12. Privacy & security

- Data minimization: collect only information needed for coordination.
- Household boundary: every object belongs to one household; cross-household access is denied by design.
- Authentication: Amazon Cognito or equivalent identity layer for deployed builds.
- Authorization: owner/member roles plus tool-level checks; agent permission never exceeds the signed-in user.
- Encryption: TLS in transit; AWS-managed encryption at rest for deployed AWS stores.
- Auditability: state-changing actions retain who/what/when and confirmation status.
- Retention: configurable; demo defaults to ephemeral synthetic data.

- Prompt injection defense: never treat arbitrary task/event text as system instructions; sanitize tool outputs and separate data from agent policy.

13. Functional requirements

ID	Requirement	Priority
FR-01	Parse one natural-language message into multiple commitments and constraints	Must
FR-02	Read current household tasks/events through tools	Must
FR-03	Detect at least time conflicts and unowned critical tasks	Must
FR-04	Generate a proposed plan with rationale	Must
FR-05	Require user confirmation for state-changing plan actions	Must
FR-06	Persist confirmed tasks/events	Must
FR-07	Show person-specific next actions	Must
FR-08	Generate a concise daily brief	Should
FR-09	Remember non-sensitive household preferences across sessions	Should
FR-10	Integrate real external calendars	Could - post-MVP

14. Non-functional requirements

- Demo determinism: seed data and scripted scenario must produce a stable result even if model wording varies.
- Latency target: first useful response within 5 seconds for demo-sized household state; stream progress where possible.
- Observability: record model/tool latency, tool failures, retries and confirmation actions.
- Graceful degradation: if Bedrock fails, show an actionable error and preserve household state.
- Accessibility: keyboard navigation, semantic labels, readable contrast and no color-only state coding.
- Portability: core domain/conflict logic remains independent from the LLM provider.

15. MVP scope

In MVP	Out of MVP / later
Web simulator for Alexa+ experience	Native mobile app
Synthetic 4-person household	Full onboarding/import from every calendar provider
Tasks, events, shopping, daily brief	Payments/purchasing
Constraint/conflict detection	Complex route optimization
Self-hosted MCP server	Production Alexa+ gated SDKs
Bedrock + Strands agent	Medical/financial decision-making
DynamoDB or local demo store	Enterprise multi-tenant admin suite
Confirmation + audit trail	Fully autonomous external actions

16. Build plan to submission

Dates (2026)	Goal	Exit criteria
Oct 2-5	Product skeleton	UI shell, synthetic family, domain model, 1 golden scenario
Oct 6-10	Agent + MCP	MCP server, 6+ tools, Strands agent, Bedrock working end-to-end
Oct 11-14	Planning quality	Conflict rules, confirmations, daily brief, stable agent prompt
Oct 15-17	AWS deployment	Hosted backend, database, logs/traces, reproducible setup
Oct 18-20	QA + polish	Tests, accessibility pass, failure states, screenshots
Oct 21	Submission assets	English demo recording, Devpost text, product feedback, friction log
Oct 22	Freeze	Fresh-environment install test, repo/license check, final video upload
Oct 23 before 12:00 PM PDT	Submit	Do not rely on last-minute upload; deadline is Oct 23, 2026 at noon Pacific

17. Testing and evaluation plan

Test type	Examples	Pass signal
Unit	Conflict overlap; ownership; due dates; timezones	Deterministic expected outputs
Tool contract	MCP schema, required fields, error codes	100% schema-valid responses
Agent scenario	Golden family scenario; ambiguous owner; impossible schedule	Correct tool selection + safe proposal
Safety	Prompt injection in task title; unauthorized household id	No policy override or cross-household data
UX	First-time user completes plan in <= 90 sec	No documentation required
Resilience	Bedrock timeout; DB unavailable; invalid tool input	Clear error, no state corruption
Demo rehearsal	Fresh browser, clean account, 3-minute script	End-to-end success 5 consecutive runs

18. Hackathon judging alignment

Criterion	How HomePilot demonstrates it
Tech Implementation	Real MCP tool execution, Strands orchestration, Bedrock reasoning, persistent state, confirmation/audit flow.
Design	Conversation-first flow, conflict cards, person-specific next actions, explainable proposals, accessible dashboard.
Potential Impact	Targets a widespread coordination burden in families and caregiving; credible path beyond hackathon.
Quality of Idea	Moves beyond a Q&A assistant or API wrapper into a stateful,

Criterion	How HomePilot demonstrates it
	multi-step household orchestration workflow.
Friction bonus	Maintain a concrete friction log during setup/integration and submit actionable feedback.
AWS Builder mini-challenge	Use a multi-service agentic architecture rather than a single model call.

19. Devpost submission copy - English

Project title

HomePilot AI - The Household Coordination Agent

Tagline

Turn family chaos into a coordinated plan - without another app everyone has to maintain.

Inspiration

Family organization tools usually assume the hard part is storing information. In reality, someone still has to notice what needs doing, remember deadlines, check everyone's availability, delegate tasks, and follow up. HomePilot AI focuses on that invisible coordination layer.

What it does

HomePilot takes a messy natural-language description of household obligations and turns it into a structured, constraint-aware plan. It can read the household context, detect conflicts, propose ownership, create tasks and events after confirmation, and generate a short daily brief for each family member. The experience is demonstrated in a web-based Alexa+ simulation and backed by a self-hosted MCP server.

How we built it

The web client sends requests to a FastAPI backend running a Strands agent. Amazon Bedrock provides model reasoning. The agent invokes household coordination tools through a self-hosted MCP server using Streamable HTTP. DynamoDB stores household state, while AWS observability services capture tool and agent execution. The demo uses deterministic synthetic household data so judges can reproduce the workflow safely.

Challenges we ran into

The hardest design problem was making the agent useful without making it over-autonomous. We separated drafting from committing: the agent can reason and propose freely, but schedule changes and external actions require explicit confirmation. We also designed tool contracts so task text is treated as data rather than instructions, reducing prompt-injection risk.

Accomplishments

A single natural-language request can trigger multi-step reasoning across people, tasks, time constraints and shopping needs; the system produces a transparent plan, asks for approval, and updates shared household state through real tool calls.

What we learned

Household coordination is less about collecting more information and more about reducing the cost of turning information into shared responsibility. Agentic workflows are especially useful when the product needs to combine context, constraints and actions across several services.

What is next

Next we would add permissioned calendar/email integrations, location-aware errands, optional Ring-based arrival events, richer household memory, and privacy-preserving personalization.

20. Product feedback template

Prompt	Draft answer / notes to capture during build
Which tools/APIs/SDKs did you use and for what?	List MCP implementation, Strands Agents, Amazon Bedrock, AgentCore if used, DynamoDB, CloudWatch, deployment services.
What worked well?	Record setup speed, docs quality, debugging visibility, tool reliability and deployment experience.
What needs work?	Capture specific errors, missing examples, confusing version requirements, local testing gaps and workarounds.
Onboarding experience	Measure time from zero to first successful model call, first agent tool call and first MCP call.
Would you build with them again?	Answer yes/no per tool with one concrete reason rather than generic praise.

21. Friction log template

Date	Task attempted	Steps	Expected	Actual	Severity	Workaround	Actionable suggestion
YYYY-MM-DD	e.g., expose MCP tool over Streamable HTTP	Commands/ config used	Expected behavior	Observed issue	Low/Med/ High	What unblocked it	Specific doc/tooling improvement

22. Three-minute demo script

Time	Visual	Narration
0:00-0:20	Messy family dashboard / chat	"Families do not fail because they lack calendars. They fail because someone still has to think for the calendar."
0:20-0:45	User enters the golden scenario	Explain that one message contains several people, deadlines and constraints.

Time	Visual	Narration
0:45-1:15	Live agent/tool trace	Show MCP calls: household context, events, conflict detection. Keep trace readable.
1:15-1:45	Plan card appears	Highlight conflict, proposed owner, rationale and travel/time buffer.
1:45-2:10	User confirms	Show real state change: tasks/events updated and audit entry created.
2:10-2:35	Person-specific views	Switch between two family members; each sees only relevant next actions.
2:35-2:50	AWS architecture overlay	Mention Bedrock + Strands + MCP + state/observability.
2:50-3:00	Closing screen	"HomePilot removes the need for one person to be the household project manager."

23. GitHub README skeleton

```
# HomePilot AI

HomePilot AI is a household coordination agent for the Amazon Developer Hackathon 2026. It
converts natural-language family context into a structured, conflict-aware plan and executes
approved actions through a self-hosted MCP server.

## Demo
- Public demo video: <URL>
- Hosted app: <URL or testing instructions>

## Track
- Primary: Alexa+
- Mini challenge: AWS Builder

## Architecture
Web simulator -> FastAPI -> Strands Agent -> Amazon Bedrock -> HomePilot MCP server -> household
tools -> DynamoDB.

## Local setup
1. Copy `.env.example` to `.env`.
2. Install Python/Node dependencies.
3. Configure AWS credentials or Bedrock API key.
4. Start MCP server.
5. Start agent/API service.
6. Start web client.
7. Load demo seed data.

## Reproducible judge scenario
Run `make demo-seed`, open the app, and paste the scenario from `demo/golden_scenario.txt`.

## Safety
State-changing actions require confirmation. Demo data is synthetic.

## License
<Choose an approved open-source license if public.>
```

24. Risk register

Risk	Impact	Mitigation
Agent output varies during demo	High	Use structured output, deterministic seed data, low temperature where available, and rule-based conflict validation.
MCP version/transport mismatch	High	Pin MCP dependency and include a startup self-test that confirms protocol version and tool listing.
AWS latency/cost	Medium	Use small demo state, stream output, set budgets, shut down idle resources, cache non-sensitive static demo context.
Too much scope	High	Freeze MVP to one golden scenario plus 2 edge cases; external calendar integration stays optional.
Privacy concerns	High	Use synthetic public demo data, explicit confirmations, clear data controls and household authorization boundaries.
Name conflict	Medium	Treat HomePilot as working title; perform final brand/trademark clearance or rename before broader release.
Submission packaging error	High	Fresh-clone test repo, verify public/private reviewer access, license, README, demo URL and all required feedback fields before deadline.

25. Definition of done

- A judge can clone the repo and run the documented demo path without private developer tooling.
- The demo visibly performs real agent/tool work rather than playing a prerecorded fake sequence.
- The MCP server exposes documented household tools and is actually invoked by the running application.
- At least one multi-constraint scenario is solved end-to-end and persisted after confirmation.
- The 3-minute public English video shows the working project and stays under the time limit.
- The submission includes product feedback, selected track(s), mini-challenge information, repository URL and testing instructions.
- The repo contains an appropriate license if public, or required reviewer access if private.
- Friction logs are concrete and actionable rather than generic complaints.

Sources and verification links

- **Amazon hackathon official rules:** https://amazonappdev2026.devpost.com/rules
- **Amazon hackathon FAQ:** https://amazonappdev2026.devpost.com/details/faqs
- **Amazon hackathon main page:** https://amazonappdev2026.devpost.com/
- **Amazon build-session resources:** https://amazonappdev2026.devpost.com/updates/46232-the-amazon-devices-build-session-recording-is-up-resources
- **Strands Agents Python quickstart:** https://strandsagents.com/docs/user-guide/sdk/quickstart/python/
- **Strands Agents MCP tools:** https://strandsagents.com/docs/user-guide/sdk/tools/mcp-tools/
- **Amazon Bedrock AgentCore - Strands integration:** https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/strands-sdk-memory.html
- **Weeks & Ruppanner (2025), cognitive labor among US parents:** https://doi.org/10.1111/jomf.13057
- **Aviv et al. (2025), cognitive household labor and wellbeing:** https://pubmed.ncbi.nlm.nih.gov/38951218/

- **Community signal: family coordination pain point:**
  https://www.reddit.com/r/homeautomation/comments/1sidwf0/